2026-10-12 题解与思路分析汇总

Tangyixiao

2026-10-12

目录

P8378 [PFOI Round1] Two Sequences 题解 **2**

P8380 Two Hypercubes 题解 **5**

P8412 「SvR-1」Hack Function! 题解 **9**

P8411 「SvR-1」Problem 题解 **14**

P2869 [USACO07DEC] Gourmet Grazers G 题解 **17**

P5021 [NOIP 2018 提高组] 赛道修建题解 **19**

P8379 Two Operations 题解 **22**

P8378 [PFOI Round1] Two Sequences 题解

思路分析

先注意题目怎么区分两种操作序列。

看的不是传给 `merge` 的两个参数，而是 `find` 以后得到的 `fx,fy`。如果两次传参不同，但找到的根一样，就不算两种方案。

题目还不允许合并已经在同一组的两个点。每次操作都减少一个组，所以 s 个点最后连成一组，恰好需要 $s-1$ 次合并。不能再额外调用一次 `merge` 来压缩路径。

把最终的 `fa` 画成树，箭头表示一个点指向自己的父亲。单独一个点不用操作，可以先不管。

比如最后要求：

$$x \to u \to r.$$

那就必须先把 x 合并到 u，再把 u 合并到 r。

如果反过来，先做 $u \to r$，那么之后 `find(u)` 找到的已经是 r，x 就会直接挂到 r 上，得不到原来要求的这条链。

所以，我们可以按从下往上的顺序，直接合并最终 `fa` 中的每一条边。下面先看哪两种形状恰好有两个顺序。

先看三个点在同一组，另外两个点都直接指向根的情况。

设最终是：

$$x \to r \leftarrow y.$$

先合并 x、再合并 y 可以；先合并 y、再合并 x 也可以。

这就是两种顺序。

如果先把 x,y 合到一起，再把它们合到 r，最终会留下 $x \to y \to r$ 或 $y \to x \to r$，而不是两个人都直接指向 r。两次合并已经用完，也没有第三次操作来压缩路径。

所以这种形状恰好对应两种操作序列。

再看两组各有两个点的情况。

设最终是两条互不相连的边：

$$x \to u, \qquad y \to v.$$

每一组内部都只需要合并一次。两组互不影响，因此先合并哪一组都行，也恰好有两种顺序。

不可能把这两组合到一起以后再拆开，所以也不会有其他方案。

剩下的形状不会恰好有两种顺序，分别看一下。

完全没有边时，只能不做任何操作，也只有一种空序列。

接着看最后有多棵需要合并的树的情况。

如果两棵树各只需要一次合并，就是上面的第二种情况。

如果一棵树至少要合并两次，另一棵树至少要合并一次，就已经有至少三种顺序。把前一棵树的两次合并记成 A_1, A_2，另一棵树的一次记成 B，即使要求 A_1 必须在 A_2 前面，也能这样排：

$$A_1, A_2, B; \qquad A_1, B, A_2; \qquad B, A_1, A_2.$$

每棵树内部的其他操作照原来的顺序做即可。

如果至少有三棵需要合并的树，仅交换这三棵树的处理顺序就已经超过两种。

于是只剩下最后只有一棵需要合并的树。

如果这棵树是一条链，前面已经说明，每条边都必须从下往上合并，顺序唯一。路径压缩只会把指针改到更高处，不会把已经跳过的父子边补回来，所以也不会多出其他顺序。

如果有分叉，就从根往下找到第一个分叉点 u。

- 如果 u 下面至少有三个孩子，先把各自的子树处理完，再交换三个孩子挂到 u 的顺序，已经超过两种。
- 如果 u 有两个孩子，其中一个孩子下面还有点，那么两边分别至少需要两次、一次合并。固定两边各自的合并顺序，仍然可以像 A_1, A_2, B 那样穿插。这些操作都发生在 u 挂到父亲之前，最终 `fa` 相同，合并顺序却至少有三种。

所以只剩一种形状：上面是一条链，下面的 u 只有两个叶子 x, y。

如果 u 本身就是根，这就是前面的三个点，已经统计过了。

如果 u 上面还有父亲 p，问题就出在路径压缩。

正常合并时，先把 x, y 分别挂到 u，再把 u 挂到 p。交换 x, y 的先后，有两种顺序。

但我们还可以这样做：

1. 执行 `merge(x,y)`，得到 $x \to y$。
2. 执行 `merge(y,u)`，得到 $x \to y \to u$。
3. 执行 `merge(x,p)`。这次 `find(x)` 会把 x, y 都直接压到 u，然后再把 u 挂到 p。

最后仍然得到：

$$x \to u \leftarrow y, \qquad u \to p.$$

如果 p 上面还有链，继续从下往上合并即可。

交换 x, y，还能再得到一种顺序。加上最开始直接合并的两种，至少有四种，因此这种形状也不能要。

这样，符合要求的确实只有前面那两种形状，其他点都必须是孤立点。

第一种，先选根，再选另外两个点：

$$n\binom{n-1}{2}.$$

第二种，先选四个点。固定其中一个点，和它配对的有 3 种选择，剩下两个点自动成一对。每一对再选哪个点当根，各有 2 种：

$$\binom{n}{4}\cdot 3\cdot 2^2 = 12\binom{n}{4}.$$

所以答案是：

$$n\binom{n-1}{2} + 12\binom{n}{4} = \frac{n(n-1)(n-2)^2}{2}.$$

模 998244353 下除以 2，乘上 2 的逆元即可。$n < 3$ 时没有符合要求的数组，直接输出 0。

每组只算一个式子，时间、额外空间复杂度都是 $O(1)$。

代码

```
namespace TANGYIXIAO {
constexpr ll mod = 998244353, iv = 499122177;
ll n, a, b, c;
inline void solve(int Task_Id) {
    cin >> n;
    if (n < 3) {
        cout << 0 << "\n";
        return;
    }
    a = n % mod, b = (n - 1) % mod, c = (n - 2) % mod;
    cout << a * b % mod * c % mod * c % mod * iv % mod << "\n";
    return;
}
} // namespace TANGYIXIAO
```

P8380 Two Hypercubes 题解

思路分析

直接枚举 x, y, z 肯定不行，范围有 10^{18}。先把等式整理一下，看看哪些数能满足。

$x = 1$ 时，y 也只能是 1，z 随便取，贡献 C。

下面考虑 $x > 1$。令 $g = \gcd(x, z)$，把 x, z 写成 ag, bg，那么 a, b 互质。原式两边同时开 g 次方根，得到：

$$y^a = x^b.$$

因为 a, b 互质，所以 x, y 一定能写成同一个底数的幂：

$$x = t^a, \qquad y = t^b.$$

比如 $a = 2, b = 3$，等式就是 $y^2 = x^3$。某个质因子在 x 里出现 r 次，在 y 里出现 s 次，就有 $2s = 3r$，所以 $r = 2k, s = 3k$。把每个质因子取 k 次乘起来，就是这个底数 t。$x = 4, y = 8$ 时，t 就是 2。

一般的 a, b 同理。这样就可以改成枚举 a, b，再数有多少个 t。$x > 1$，所以 t 至少是 2。

由 $x = ag = t^a$，还能算出：

$$z = bg = \frac{bt^a}{a}.$$

z 必须是整数，因此 a 要能整除 bt^a。a, b 互质，这个条件就是 a 能整除 t^a。

把 a 的不同质因子乘起来，记成 d，那么 t 必须是 d 的倍数。

比如 $a = 12 = 2^2 \cdot 3$，$d = 2 \cdot 3 = 6$。t 只要同时含有 2, 3 就够了，因为乘 12 次以后，这两个质因子的次数肯定够用。反过来，t 如果缺少其中一个质因子，乘多少次也补不出来。

a, b 定下来以后，t 越大，x, y, z 都越大。分别根据 A, B, C 算出 t 最多能取多少，再取最小值，记成 R。

举个例子，$a = 2, b = 3$，$A = 100, B = 1000, C = 96$。

$x = t^2 \le 100$，所以 $t \le 10$；$y = t^3 \le 1000$，也要求 $t \le 10$。

$z = 3t^2/2 \le 96$，也就是 $t^2 \le 64$，要求 $t \le 8$。

三个条件一起满足，就只能取到 8。又因为 $a = 2$，t 必须是偶数，所以只有 2, 4, 6, 8，这一组 a, b 贡献 4。

一般情况下也是这样。$a > 1$ 时，$d \ge 2$，合法底数就是 $d, 2d, 3d, \ldots$，不超过 R 的一共有 $\lfloor R/d \rfloor$ 个。

$a = 1$ 时，$d = 1$，但 $t = 1$ 已经在开头统计过，所以只数 2 到 R，贡献 $\max(R - 1, 0)$。

计算上界时，前两个条件分别求 A 的整数 a 次方根、B 的整数 b 次方根。第三个条件先整理成 $t^a \le \lfloor aC/b \rfloor$，再求整数 a 次方根。三个结果取最小即可。

代码里的 `rt(x,k)` 就是求最大的整数 t，使 $t^k \le x$。算出候选值后，用整数快速幂检查边界；中间计算用 128 位整数，幂超过上界后就不必继续算出完整结果。

$t \ge 2$，而 2^{60} 已经超过 10^{18}，所以 a, b 都不会超过 59。枚举所有互质的 a, b，把上面的贡献加起来即可。每个三元组的 a, b 由 $\gcd(x, z)$ 唯一确定，t 也由 $x = t^a$ 确定，不会重复统计。

每组枚举 $O(\log^2 V)$ 个指数对，其中 $V = 10^{18}$。计入整数根计算，时间复杂度 $O(\log^2 V \log\log V)$，空间复杂度 $O(\log^2 V)$。

代码

```
namespace TANGYIXIAO {
const int N = 6e1, M = 4e3 + 5;
const ll mod = 1e9 + 7;
ull A, B, C, ra[N], rb[N];
int pa[M], pb[M], pc;
ll ans;
inline u128 pw(ull x, int e, u128 lim) {
    u128 v = 1, b = x;
    for (; e;) {
        if (e & 1) {
            if (b > lim / v) {
                return lim + 1;
            }
            v *= b;
        }
        e >>= 1;
        if (e) {
            if (b > lim / b) {
                b = lim + 1;
            } else {
                b *= b;
            }
        }
    }
    return v;
}
inline bool chk(ull x, int e, u128 lim) {
    return pw(x, e, lim) <= lim;
}
inline ull rt(u128 x, int e) {
    if (x == 0) {
        return 0;
    }
    if (e == 1) {
        return (ull)x;
    }
    int k = 0;
    for (u128 v = x; v; v >>= 1) {
        k++;
    }
```

```
    ull r = 1ULL << ((k + e - 1) / e);
    for (;;) {
        u128 p = pw(r, e - 1, x);
        u128 q = p > x ? 0 : x / p;
        ull nr = (ull)(((u128)(e - 1) * r + q) / e);
        if (nr >= r) {
            break;
        }
        r = nr;
    }
    for (; chk(r + 1, e, x);) {
        r++;
    }
    for (; !chk(r, e, x);) {
        r--;
    }
    return r;
}
inline int rd(int x) {
    int r = 1;
    for (int p = 2; p * p <= x; p++) {
        if (x % p == 0) {
            r *= p;
            for (; x % p == 0;) {
                x /= p;
            }
        }
    }
    if (x > 1) {
        r *= x;
    }
    return r;
}
inline void ini() {
    if (pc) {
        return;
    }
    for (int a = 1; a < N; a++) {
        for (int b = 1; b < N; b++) {
            if (MATH::gcd(a, b) == 1) {
                pa[pc] = a, pb[pc] = b, pc++;
            }
        }
    }
    return;
}
inline void solve(int Task_Id) {
    cin >> A >> B >> C;
    ini();
    for (int e = 1; e < N; e++) {
        ra[e] = rt(A, e), rb[e] = rt(B, e);
```

```
    }
    ans = C % mod;
    for (int i = 0; i < pc; i++) {
        int a = pa[i], b = pb[i];
        int d = rd(a), mn = (a == 1 ? 2 : d);
        if (ra[a] < (ull)mn || rb[b] < (ull)mn) {
            continue;
        }
        u128 lim = (u128)C * a / b;
        ull r = min(rb[b], lim < (u128)A ? rt(lim, a) : ra[a]);
        ull cnt = a == 1 ? (r > 1 ? r - 1 : 0) : r / d;
        ans = (ans + cnt % mod) % mod;
    }
    cout << ans << "\n";
    return;
}
} // namespace TANGYIXIAO
```

P8412「SvR-1」Hack Function! 题解

思路分析

先看十进制：538 的数位和是 $5+3+8=16$，再算一次得到 $1+6=7$。原数 538、数位和 16、最后的 7，除以 9 的余数都是 7。

原因是 10 除以 9 余 1。把一个数写成各位数字乘 10 的幂后，每个 10^j 除以 9 都余 1，所以原数和数位和除以 9 的余数相同。换成 k 进制也一样：k 除以 $k-1$ 余 1，每做一次数位和，模 $k-1$ 的余数都不变。

令 $q=k-1$。反复做数位和后，结果会落在 1 到 q 之间。因此只要知道 $A \bmod q$，就知道 $f(A)$：余数非零时答案就是这个余数；余数为零时答案是 q。题目里的数是 i^i，所以接下来只需研究 $i^i \bmod q$。

接下来找数列的周期。

记 $\varphi(q)$ 为欧拉函数，也就是 1 到 q 中与 q 互质的数的个数。取：

$$P=q\varphi(q).$$

从 $i=q$ 开始，$f(i^i)$ 每隔 P 项重复一次。

先看为什么。把 q 按质因数拆开，比如 $q=12$，就分别看模 4 和模 3 的结果。一般把其中一块记成 s^e，其中 s 是质数。

如果 s 能整除 i，那么 i^i 至少含有 i 个因子 s。由于 $i \geq q \geq s^e \geq e$，这些因子足够让 s^e 整除 i^i，所以余数是 0。

P 是 q 的倍数，因此 $i+P$ 也能被 s 整除，指数同样足够大，余数还是 0。

如果 s 不能整除 i，那 i 与 s^e 互质。这时可以用欧拉定理：指数每增加 $\varphi(s^e)$，幂模 s^e 的余数不变。

$i+P$ 与 i 模 s^e 的余数相同，所以只需要看指数多出的 P。$\varphi(q)$ 是各个 $\varphi(s^e)$ 的乘积，P 也是 $\varphi(s^e)$ 的倍数，因此余数仍然相同。

这样，在每个 s^e 下，两项的余数都相同。各个素数幂互质，合起来模 q 的余数也相同。比如一个差同时能被 $4,3$ 整除，也就能被 12 整除。

周期长度不到一百万，可以直接存下一个周期。固定 i 除以 q 的余数 x。周期中对应的下标依次是 $q+x$，$q+x+q$，$q+x+2q,\ldots$。这些数模 q 都是 x，指数每次增加 q，所以幂的余数按下面的式子递推：

$$(q+x+tq)^{q+x+tq} \equiv x^{q+x}(x^q)^t \pmod q.$$

让 x 从 0 到 $q-1$，依次填入下标 $x, x+q, x+2q, \ldots$，就能覆盖整个周期。每个 x 只需算出第一项和递推时要乘的数；余数为 0 的位置按前面的规则记作 q。

有了周期，再来找和为 p 的区间。

周期前还有 1 到 $q-1$ 这些项，区间可能只在前面、只在周期里，也可能跨过两段。

先看区间完全在前 $q-1$ 项里的情况。

这一段最多只有 $q-1 \le 998$ 项。枚举左右端点并累加；和等于 p 时输出。若和已经超过 p，就停止向右扩展，因为每项都是正数，继续加只会更大。

如果区间从周期里开始，就可以先取完整的周期。

记一个周期的和为 C。无论从周期的哪个位置开始，连续走完整个周期，和都是 C。

所以先用除法把目标拆成：

$$p = cC + t, \qquad 0 \le t < C.$$

意思是先取 c 个完整周期，再凑余下的 t。

假设一个周期是 $2,3,4$，周期和为 9。要凑 $23 = 2 \cdot 9 + 5$，就先取两个完整周期，再接 $2,3$ 这两项。

完整周期走完以后又回到同一个位置，所以接上的部分仍然连续。

$t = 0$ 时，直接取 c 个周期就行。否则要在周期数组里找一段和为 t 的连续段；走到数组末尾后，可以绕回开头。

这段长度一定小于 P，因为走满 P 项，和就已经是 $C > t$。

各项都是正数，可以用双指针：和不够，就向右加一项；和超过了，就从左边减掉一项。两个指针只向右走，不需要回头。

找到起点和长度后，就从这个位置先走 c 个周期，再接上这段。

还要考虑从前 $q-1$ 项开始，一直延伸到周期里的情况。

若起点是 $l < q$，区间到达周期前，必须先完整经过 l 到 $q-1$。代码用 `sf[l]` 存这段的和。

如果这一段的和已经超过 p，这个起点不可能。否则，剩下要凑的和是 $p - sf[l]$。

后面的周期部分固定从下标 0 开始。先取若干完整周期，余下的只能是周期开头的若干项。

因此，枚举 l，先用除法算出完整周期数，再在周期的前缀和数组里二分，看看有没有刚好等于余下目标和的位置。

周期各项都是正数，前缀和严格递增，所以可以二分。找到后，根据总共取了多少项算出右端点即可。

以上三种情况覆盖了所有区间起点。都找不到时，输出 -1 -1。

每个 $f(i^i)$ 至少为 1，所以和为 p 的区间最多有 p 项。题目给出 $p \le 10^8$，故区间长度不超过 p，而 $r-l$ 比项数还少 1，满足题面范围。构造出的周期起点不超过 $q+P-1$，约一百万；终点再加上至多 $p-1$，也远小于 10^{18}。

设 $P = q\varphi(q)$。每组时间复杂度 $O(P+q^2)$，空间复杂度 $O(P)$。

由于 $q \le 999$，周期长度不超过 $999 \cdot 998 = 997002$，所以 `b`、`ps` 开到 `N = 1e6 + 5` 即可。

`a` 存前 $q-1$ 项，`sf[l]` 存 l 到 $q-1$ 的和，两者开到 `K = 1e3 + 5` 即可。

代码

代码只列出模板末尾的 `namespace TANGYIXIAO` 部分。

```
namespace TANGYIXIAO {
using ll = long long;
const int N = 1e6 + 5, K = 1e3 + 5;
int a[K], b[N];
ll ps[N], sf[K];
int k, q, ph, m, st, len;
ll p, cs, c, t, nd, cc, rem, tot;

inline ll pw(ll x, int e, int y) {
    ll r = 1 % y;
    for (; e; e >>= 1) {
        if (e & 1) {
            r = r * x % y;
        }
        x = x * x % y;
    }
    return r;
}

inline int phi(int x) {
    int r = x;
    for (int d = 2; d * d <= x; d++) {
        if (x % d == 0) {
            r = r / d * (d - 1);
            for (; x % d == 0; x /= d) {
            }
        }
    }
    if (x > 1) {
        r = r / x * (x - 1);
    }
    return r;
}

inline bool chk(int *a, int n, ll v, int &st, int &len) {
    if (v <= 0) {
        return false;
    }
    int r = 0;
    ll sum = 0;
    for (int i = 0; i < n; i++) {
        for (; r < i + n && sum < v; r++) {
            sum += a[r % n];
        }
        if (sum == v) {
            st = i, len = r - i;
            return true;
        }
```

```
        if (r > i) {
            sum -= a[i];
        }
    }
    return false;
}

inline void out(ll l, ll r) {
    cout << l << "␣" << r << "\n";
    return;
}

inline void solve(int Task_Id) {
    cin >> k >> p;
    q = k - 1, ph = phi(q), m = q * ph;
    ps[0] = 0, sf[q] = 0;
    for (int i = 1; i < q; i++) {
        int x = (int)pw(i, i, q);
        a[i] = x ? x : q;
    }
    for (int x = 0; x < q; x++) {
        if (x == 0) {
            for (int t = 0; t < ph; t++) {
                b[t * q] = q;
            }
            continue;
        }
        ll v = pw(x, q + x, q), r = pw(x, q, q);
        for (int t = 0; t < ph; t++) {
            int j = x + t * q;
            b[j] = v ? (int)v : q;
            v = v * r % q;
        }
    }
    cs = 0;
    for (int i = 0; i < m; i++) {
        cs += b[i];
    }

    for (int l = 1; l < q; l++) {
        ll sum = 0;
        for (int r = l; r < q; r++) {
            sum += a[r];
            if (sum == p) {
                out(l, r);
                return;
            }
            if (sum > p) {
                break;
            }
        }
```

```
    }

    c = p / cs, t = p % cs;
    if (t == 0 && c > 0) {
        out(q, q + c * m - 1);
        return;
    }
    if (chk(b, m, t, st, len)) {
        out(q + st, q + st + c * m + len - 1);
        return;
    }

    for (int i = 0; i < m; i++) {
        ps[i + 1] = ps[i] + b[i];
    }
    for (int i = q - 1; i >= 1; i--) {
        sf[i] = sf[i + 1] + a[i];
    }
    for (int l = 1; l < q; l++) {
        if (p < sf[l]) {
            continue;
        }
        nd = p - sf[l], cc = nd / cs, rem = nd % cs;
        ll *it = lower_bound(ps, ps + m, rem);
        if (it == ps + m || *it != rem) {
            continue;
        }
        tot = cc * m + (it - ps);
        if (tot > 0) {
            out(l, q - 1 + tot);
            return;
        }
    }

    out(-1, -1);
    return;
}
} // namespace TANGYIXIAO
```

P8411「SvR-1」Problem 题解

思路分析

原先用大根堆，每次做当前已经解锁的题里 a_i 最大的一道。这个贪心可以得到正确答案，但时间复杂度是 $O(n\log n)$；n 达到 10^7 时，维护堆的开销过大，提交只得到 50 分。

这道题只要求最大总和，不要求输出做题顺序。可以先求出每道题的贡献上限，再证明这些上限能同时达到。

每道题的贡献上限

设 b_i 为从根 1 到节点 i 的路径上，所有 a 的最小值。

做第 i 道题之前，必须已经做完它的所有祖先。因此做完它时，高兴程度一定不超过 b_i，任何合法方案的答案都满足

$$\text{ans} \le \sum_{i=1}^{n} b_i.$$

例如，一条路径上的 a 依次为 $10, 3, 12$，那么这三个节点的 b 依次为 $10, 3, 3$。最后一道题虽然有趣程度是 12，但做它之前必须经过值为 3 的祖先，所以它的贡献最多是 3。

这些上限可以同时达到

考虑按 b_i 从大到小做题；b_i 相同时，按节点编号从小到大做。

对于每条父子边，有

$$b_{fa_i} \ge b_i.$$

如果父亲的 b 更大，它一定先做；如果相同，由于 $fa_i < i$，父亲仍然先做。因此这个顺序合法，并且第 1 道题一定排在最前面。

轮到节点 i 时，之前做过的每个节点 j 都满足 $b_j \ge b_i$。又因为 $a_j \ge b_j$，所以之前做过的题都不会把高兴程度降到 b_i 以下，当前题也满足 $a_i \ge b_i$。

另一方面，根到 i 的路径上必定有一个节点的 a 等于 b_i。它是 i 自己或已经做过的祖先，所以做完 i 时，高兴程度也不可能超过 b_i。

两者结合，做完第 i 道题后的高兴程度恰好为 b_i。于是上界可以达到，答案就是

$$\boxed{\text{ans} = \sum_{i=1}^{n} b_i}.$$

这里只用排序顺序证明答案存在，代码不需要排序，也不需要维护堆。

线性计算

路径最小值满足

$$b_1 = a_1, \qquad b_i = \min(a_i, b_{fa_i}) \quad (i \geq 2).$$

题目保证 $fa_i < i$，按编号从小到大处理时，父亲的 b 已经求出。因此可以直接把数组 `a` 改成路径最小值，不需要另开数组。

实现分两遍：

1. 严格按题目的随机数生成顺序，先生成全部原始 a_i。
2. 令答案为 a_1，再依次生成 fa_i，执行 `a[i] = min(a[i], a[fa_i])`，把更新后的 `a[i]` 加入答案。

不能把生成 a_i 和生成 fa_i 交错执行，否则随机数序列会改变，得到的就不是题目规定的数据。

样例中，各节点的路径最小值依次为

$$3398922311,\ 3077554952,\ 2933028207,\ 3398922311,\ 1263042788,\ 835814542.$$

相加得到 14907285111。

时间复杂度 $O(n)$，空间复杂度 $O(n)$。只使用一个 `uint32_t` 数组，$n = 10^7$ 时数组约占 38.15 MiB。随机数状态和数组元素必须保留 32 位无符号类型，以正确处理移位、溢出以及大于 $2^{31} - 1$ 的数值。

答案最多为 $10^7(2^{32} - 1) < 2^{63} - 1$，用 `long long` 保存。$n = 1$ 时直接输出 a_1；$seed = 0$ 时随机数始终为 0，答案也为 0。

代码

代码只列出模板末尾的 `namespace TANGYIXIAO` 部分。

```
namespace TANGYIXIAO {
const int N = 1e7 + 5;
uint32_t a[N], sd;
int n;
long long ans;

inline uint32_t rnd() {
    sd ^= sd << 13;
    sd ^= sd >> 17;
    sd ^= sd << 5;
    return sd;
}

inline void solve(int Task_Id) {
    cin >> n >> sd;
    for (int i = 1; i <= n; i++) {
        a[i] = rnd();
```

```
    }
    ans = a[1];
    for (int i = 2; i <= n; i++) {
        a[i] = min(a[i], a[rnd() % (i - 1) + 1]), ans += a[i];
    }
    cout << ans << "\n";
    return;
}
} // namespace TANGYIXIAO
```

P2869 [USACO07DEC] Gourmet Grazers G 题解

思路分析

每头牛要买一份草。草的价格和口感都要达标，而且一份草只能给一头牛。先处理口感要求最高的牛。

把牛按口感要求从高到低排好，草也按口感从高到低排好。处理一头牛时，把所有口感够的草放进候选池。后面的牛口感要求更低，所以池里的草也都能满足它们的口感要求。

比如当前牛要求口感至少 8、价格至少 5，池里有三份草：$(4,10)$、$(7,9)$、$(9,8)$，依次表示价格和口感。

第一份价格不够；另外两份都能用，所以选价格为 7 的草。

为什么能选价格最低的合格草?

把贪心选中的草叫 h。先看一个最优方案：如果当前牛原本拿到的是另一份草 g，那么 g 也在候选池里，并且价格不低于当前要求。因此贪心选出的 h 价格不高于 g。

如果最优方案把 h 留着没用，就让当前牛改用 h，花费不会增加。

如果 h 原本分给了后面的一头牛，就交换两份草：当前牛拿 h，后面那头牛拿 g。

g 的口感不低于当前牛的要求，也就不低于后面那头牛的要求。

价格方面，后面那头牛原本能用 h，而 g 的价格不低于 h，所以 g 的价格要求也一定达标。

交换后两头牛都能吃到合格的草，总花费也不变。

用 `multiset` 按价格保存候选池。对当前牛从它的价格下限开始找第一份草，也就是池中最便宜的合格草。如果找不到，当前牛没有可用的草，输出 -1。

排序和集合操作的总复杂度为 $O((n+m)\log(n+m))$，空间复杂度为 $O(n+m)$。

代码

```
namespace TANGYIXIAO {
const int N = 1e5 + 5;
int n, m, j;
ll ans;
multiset<ll> s;
struct C {
    ll x, y;
} a[N];
struct G {
    ll x, y;
} b[N];
inline bool cc(const C &x, const C &y) { return x.y > y.y; }
inline bool cg(const G &x, const G &y) { return x.y > y.y; }
inline void solve(int Task_Id) {
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        cin >> a[i].x >> a[i].y;
```

```
    }
    for (int i = 1; i <= m; i++) {
        cin >> b[i].x >> b[i].y;
    }
    sort(a + 1, a + n + 1, cc);
    sort(b + 1, b + m + 1, cg);
    s.clear(), ans = 0, j = 1;
    for (int i = 1; i <= n; i++) {
        for (; j <= m && b[j].y >= a[i].y; j++) {
            s.insert(b[j].x);
        }
        auto it = s.lower_bound(a[i].x);
        if (it == s.end()) {
            cout << -1 << "\n";
            return;
        }
        ans += *it;
        s.erase(it);
    }
    cout << ans << "\n";
    return;
}
} // namespace TANGYIXIAO
```

P5021 [NOIP 2018 提高组] 赛道修建题解

思路分析

先猜一个长度 L，只问能不能修出至少 m 条长度都不小于 L 的赛道。若 L 可行，更小的长度也一定可行，所以可以二分 L。

把 1 当作根，从下往上处理。已经凑好的赛道就放在子树里，剩下还没凑够长度的链才可能继续往上接。

为什么一个点最多往父亲留一条链？因为它到父亲只有一条边，所有往上走的链都必须经过这条边，而这条边只能给一条赛道用。

这条链从 u 出发，沿着孩子的子树走到某个端点。设孩子 v 留下的链长为 f_v，连接 u, v 的边长为 $w(u, v)$，那么这条链到 u 为止的长度是

$$x_v = f_v + w(u, v).$$

如果 $x_v \geq L$，从这条链的端点走到 u，已经是一条够长的赛道，可以直接记一条。再和别的短链拼，只会多用一条链，并不会多得到一条赛道，所以不必继续拼。

剩下的链都短于 L。在 u 这里，取两条不同孩子留下的链就能拼成一条赛道。比如 $L = 10$，链长是 $2, 3, 7, 8$：最短的 2 需要找一条至少 8 长的链，于是和 8 配成一条；接下来 3 和 7 配成一条。

每次都拿最短的链 x，再找最短的一条链 y，让 $x + y \geq L$。

为什么不用更长的搭档？看一个配对数最多的方案。假设 x 和更长的 z 配对，y 和 t 配对，可以交换成：

$$(x, z), (y, t) \longrightarrow (x, y), (z, t).$$

$x + y$ 本来就够长。又因为 $z \geq y$，原来够长的 $y + t$ 换成 $z + t$，只会更长。因此两条赛道都保住了。

如果 y 原本没配对，那改成 $x + y$ 后，留下的是更长的 z。

如果 x 原本没配对，y 和 t 配对，那改成 $x + y$ 后留下 t。x 是最短的链，所以 $t \geq x$，留下的也不会更短。

如果 x, y 都没配对，直接配起来就能多出一条赛道，与“配对数已经最多”矛盾。

所以这个贪心能保住最多的赛道数，而且不会把能留给父亲的最长链变短。

如果最短链 x 找不到搭档，那么剩下的链都小于 $L - x$，没有一条能和它拼够 L。把它记作一个向上保留的候选，再处理下一条链即可。所有没配上的链中，只把最长的一条交给父亲。因为只能留一条，当然留越长的越容易在上面凑够 L。

根节点处理完后，数一数已经凑出的赛道是否至少有 m 条，就能判断 L 是否可行。二分上界可以取所有边长之和除以 m，因为 m 条互不共用道路的赛道，总长度不可能超过整棵树的边长总和。

一次判断中，每条边最多进出一次有序集合，时间复杂度为 $O(n \log n)$。二分后总时间复杂度为 $O(n \log n \log W)$，其中 W 是所有边长之和；空间复杂度为 $O(n)$。

代码用前向星存边。先从根节点迭代遍历，记下每个点的父节点和遍历顺序；判定时倒着处理这个顺序，就能先算完孩子再算父亲，也不需要递归遍历树。

代码

```
namespace TANGYIXIAO {
const int N = 5e4 + 5, M = 1e5 + 5;
struct E {
    int v, ne;
    ll w;
};
int n, m, h[N], fa[N], ord[N], k, ed, u, v, cnt;
E e[M];
ll f[N], pw[N], sum, l, r, mid, w, res;
multiset<ll> s;
inline void add(int u, int v, ll w) { e[++ed] = {v, h[u], w}, h[u] = ed; }
inline bool chk(ll lim) {
    cnt = 0;
    for (int i = n; i; i--) {
        int u = ord[i];
        s.clear();
        for (int j = h[u]; j; j = e[j].ne) {
            int v = e[j].v;
            if (fa[v] != u) {
                continue;
            }
            ll x = f[v] + pw[v];
            if (x >= lim) {
                cnt++;
                if (cnt >= m) {
                    return true;
                }
            } else {
                s.insert(x);
            }
        }
        res = 0;
        for (; !s.empty();) {
            auto it = s.begin();
            ll x = *it;
            s.erase(it);
            auto jt = s.lower_bound(lim - x);
            if (jt == s.end()) {
                res = max(res, x);
            } else {
                s.erase(jt);
                cnt++;
                if (cnt >= m) {
                    return true;
                }
```

```
            }
        }
        f[u] = res;
    }
    return cnt >= m;
}
inline void solve(int Task_Id) {
    cin >> n >> m;
    for (int i = 1; i < n; i++) {
        cin >> u >> v >> w;
        add(u, v, w);
        add(v, u, w);
        sum += w;
    }
    k = 1;
    ord[1] = 1;
    for (int i = 1; i <= k; i++) {
        int u = ord[i];
        for (int j = h[u]; j; j = e[j].ne) {
            int v = e[j].v;
            if (v == fa[u]) {
                continue;
            }
            fa[v] = u;
            pw[v] = e[j].w;
            ord[++k] = v;
        }
    }
    l = 0, r = sum / m + 1;
    for (; l + 1 < r;) {
        mid = (l + r) / 2;
        if (chk(mid)) {
            l = mid;
        } else {
            r = mid;
        }
    }
    cout << l << "\n";
    return;
}
} // namespace TANGYIXIAO
```

P8379 Two Operations 题解

思路分析

先只考虑一次 `Query`。

我们可以决定最后把大家合并进哪一组。设选的是第 i 组，先把其他组合到一起，最后再整体合并进它。

记所有学生的糖果总数为 C，第 i 组的糖果总数为 s_i，最多糖果数为 mx_i，总人数为 n。

最后一次合并时，其他组的学生先被清零，然后所有人都得到：

$$\frac{C-s_i}{n}$$

颗糖。

第 i 组原来最多有 mx_i 颗糖，所以最后的最大值就是：

$$mx_i+\frac{C-s_i}{n}.$$

比如样例里，一组有 $3,4$ 颗糖，另一组有 5 颗糖。把第二组合并进第一组，三个人都增加 $5/3$，原来第一组的 4 颗糖保留，最终最大值就是：

$$4+\frac{5}{3}=\frac{17}{3}.$$

分几次合并也不会更好。

无论怎么合并，总有一组从头到尾留到最后。它可以接收别的组，但不能被合并进别人，否则这一组就被删了。

设这一组是 i。它原来的学生从来没有被清零，所以 mx_i 一直保留。

其余的 $C-s_i$ 颗糖，可能分几批合并进来。每一批分摊时，组内人数都不超过 n，所以组 i 原来的每个人收到的糖果，不会少于把这一批直接除以 n。

加起来，至少增加 $(C-s_i)/n$。

因此，先把其他组合成一组、最后一次性合并进 i，就已经最好了。剩下只要选出最合适的 i。

把式子整理一下：

$$mx_i+\frac{C-s_i}{n}=\frac{C+(nmx_i-s_i)}{n}.$$

C,n 都不会改变，所以只要维护：

$$\min_i(nmx_i-s_i).$$

用 `set` 存每组的 $(nmx_i - s_i, i)$。带上组号，是因为不同组算出的值可能一样，删除时要能区分。

`Change` 只改组别，每个人的糖果数保持原值。清零、平均分糖，是 `Query` 计算答案时假设进行的操作，不会真的发生。

所以 `Change X Y` 时：

1. 把 X, Y 原来的值从集合中删掉。
2. 糖果总数相加，最大值取两组最大值的较大者。
3. 删除 X，把更新后的 Y 放回集合。

`Query` 时取集合中的最小值，代回上面的式子即可。最后除以 n，用模 $10^9 + 7$ 下的逆元处理。

初始化 $O(n + k \log k)$，每次修改 $O(\log k)$，每次询问 $O(1)$，空间 $O(n + k)$。

代码

```
namespace TANGYIXIAO {
const int N = 1e6 + 5, M = 1e1;
const long long MOD = 1e9 + 7;
int n, m, k, g[N], x, y;
long long s[N], mx[N], tot, iv, v;
char op[M];
set<pair<long long, int>> st;
inline long long pw(long long a, long long b) {
    long long r = 1;
    for (; b; b >>= 1, a = a * a % MOD) {
        if (b & 1) {
            r = r * a % MOD;
        }
    }
    return r;
}
inline long long get(int x) { return 1LL * n * mx[x] - s[x]; }
inline void solve(int Task_Id) {
    cin >> n >> m >> k;
    for (int i = 1; i <= n; i++) {
        cin >> g[i];
    }
    for (int i = 1; i <= n; i++) {
        cin >> v;
        s[g[i]] += v, mx[g[i]] = max(mx[g[i]], v), tot += v;
    }
    for (int i = 1; i <= k; i++) {
        st.emplace(get(i), i);
    }
    iv = pw(n, MOD - 2);
    for (int i = 1; i <= m; i++) {
        cin >> op;
        if (op[0] == 'Q') {
            cout << (tot + st.begin()->first) % MOD * iv % MOD << "\n";
```

```
        } else {
            cin >> x >> y;
            if (x == y) {
                continue;
            }
            st.erase({get(x), x}), st.erase({get(y), y});
            s[y] += s[x], mx[y] = max(mx[y], mx[x]);
            s[x] = mx[x] = 0;
            st.emplace(get(y), y);
        }
    }
    return;
}
} // namespace TANGYIXIAO
```